Bazel

Bazel for C++ and Python

NobleProg

DELIVERY MODE

- 8 Hours Daily of Instructor-Led Training
- Slides Based Content
- Whiteboard/ Pen Tab based Interaction
- Hands-on Live Demo
- Daily Recap
- Animated Quiz for Quik recap

PRE-REQUISITES

- Basic Knowledge of C++
- Basic Knowledge of Python
- Basic Knowledge of Building of A Project
- Basic Knowledge of IDE
- Basic Knowledge of Project Architecture

Bazel Overview

Bazel is:

An open-source build and test tool similar to Make, Maven, and Gradle.

It uses a human-readable, high-level build language.

Bazel supports projects in multiple languages and builds outputs for multiple platforms.

Bazel supports large codebases across multiple repositories, and large numbers of users.

Bazel Benefits

- **High-level build language.** Bazel uses an abstract, human-readable language to describe the build properties of your project at a high semantical level.

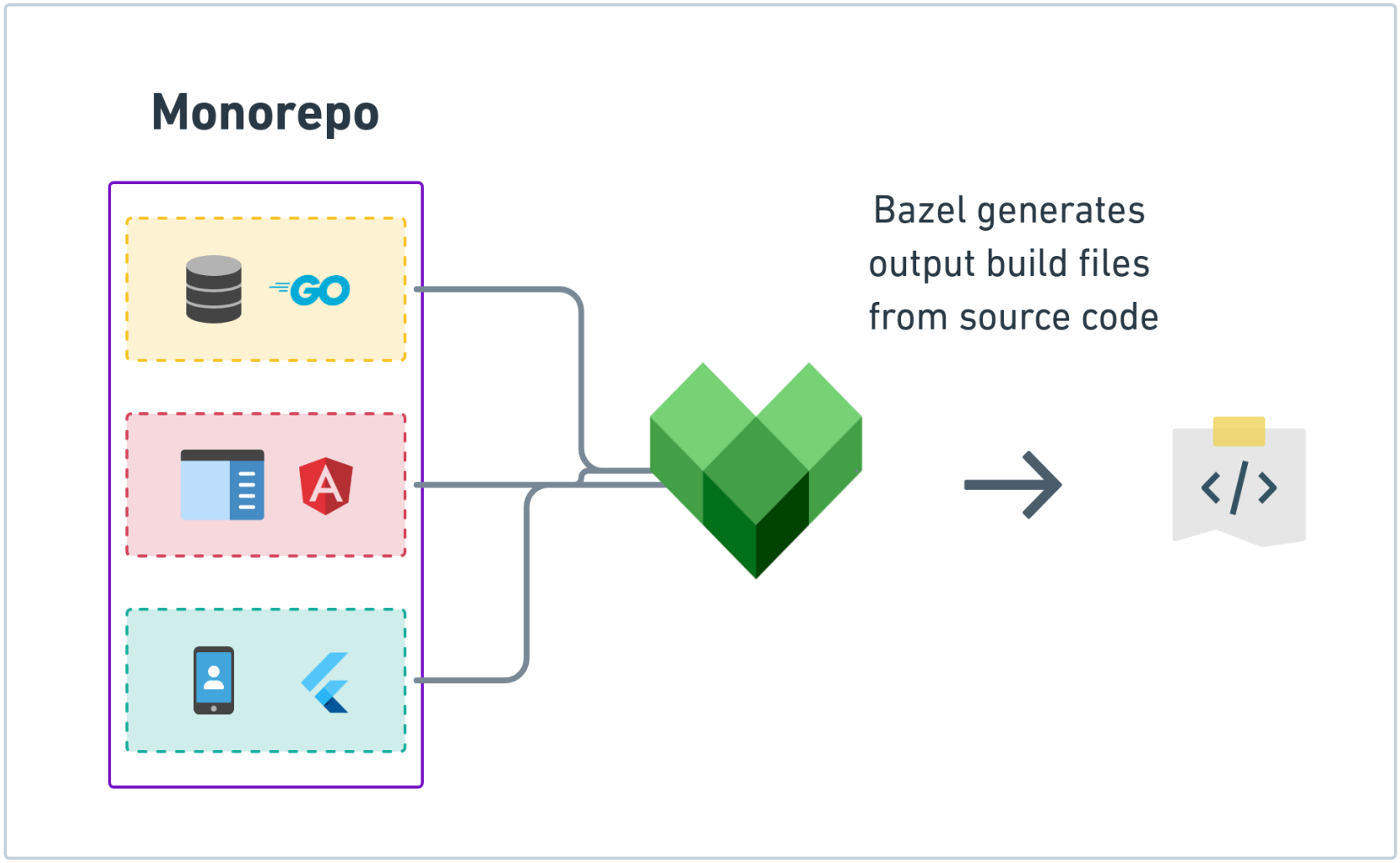
- **Bazel is fast and reliable.** Bazel caches all previously done work and tracks changes to both file content and build commands. This way, Bazel knows when something needs to be rebuilt, and rebuilds only that. To further speed up your builds, you can set up your project to build in a highly parallel and incremental fashion.

- **Bazel is multi-platform.** Bazel runs on Linux, macOS, and Windows. Bazel can build binaries and deployable packages for multiple platforms, including desktop, server, and mobile, from the same project.

- **Bazel scales.** Bazel maintains agility while handling builds with 100k+ source files. It works with multiple repositories and user bases in the tens of thousands.

- **Bazel is extensible.** Many languages are supported, and you can extend Bazel to support any other language or framework.

Bazel Monorepo



Bazel Terminology

- WORKSPACE/MODULE.bazel
- Package
- BUILD file
- Target
- Labels
- Dependencies
- Commands

Bazel Terminology contd..

WORKSPACE/MODULE.bazel

The workspace is typically the directory where Bazel constructs your project's build files from your source code. It contains various source files in a nested hierarchical fashion. At the highest or root level of your workspace, your project might also have a dedicated WORKSPACE text file. It contains references to all the external dependencies your project requires to generate your builds. Precisely, the workspace is where your inputs are extracted and converted to outputs to generate the desired build file.

WORKSPACE

```
load("@bazel_tools//tools/build_defs/repo:http.bzl", "http_archive")

RULES_JVM_EXTERNAL_TAG = "2.10"
RULES_JVM_EXTERNAL_SHA = "1bbf2e48d07686707dd85357e9a94da775e1dbd7c464272b3664283c9c716d26"

http_archive(
    name = "rules_jvm_external",
    strip_prefix = "rules_jvm_external-%s" % RULES_JVM_EXTERNAL_TAG,
    sha256 = RULES_JVM_EXTERNAL_SHA,
    url = "https://github.com/bazelbuild/rules_jvm_external/archive/%s.zip" % RULES_JVM_EXTERNAL_TAG,
)

load("@rules_jvm_external//:defs.bzl", "maven_install")
```

Bazel Terminology contd..

Packages

Packages are simply directories located below the top-level directory in a workspace. They contain your build files that may be named BUILD or BUILD.bazel , along with other related files and specified dependencies. One package may be nested in another since your source code is organized in a hierarchical fashion. Consider the following working directory:

```
src/application/BUILD
src/application/main/render.txt
src/application/staging/BUILD
```

The application is a package as it contains its own BUILD file. Similarly, staging is another package but since it falls directly under application, it can be considered as a *subpackage*. However, the main is not a subpackage but only a regular directory inside your application package since it doesn't have its own BUILD file.

Bazel Terminology contd..

BUILD File

Build files contain the program that enables Bazel to generate the desired builds for your project. It's evaluated using Starlark and contains a list of sequential statements that are executed by Bazel. The build files contain your declared rules, functions that execute these rules, and variables required in a simple syntax form. Under a WORKSPACE every package contains its own BUILD file.

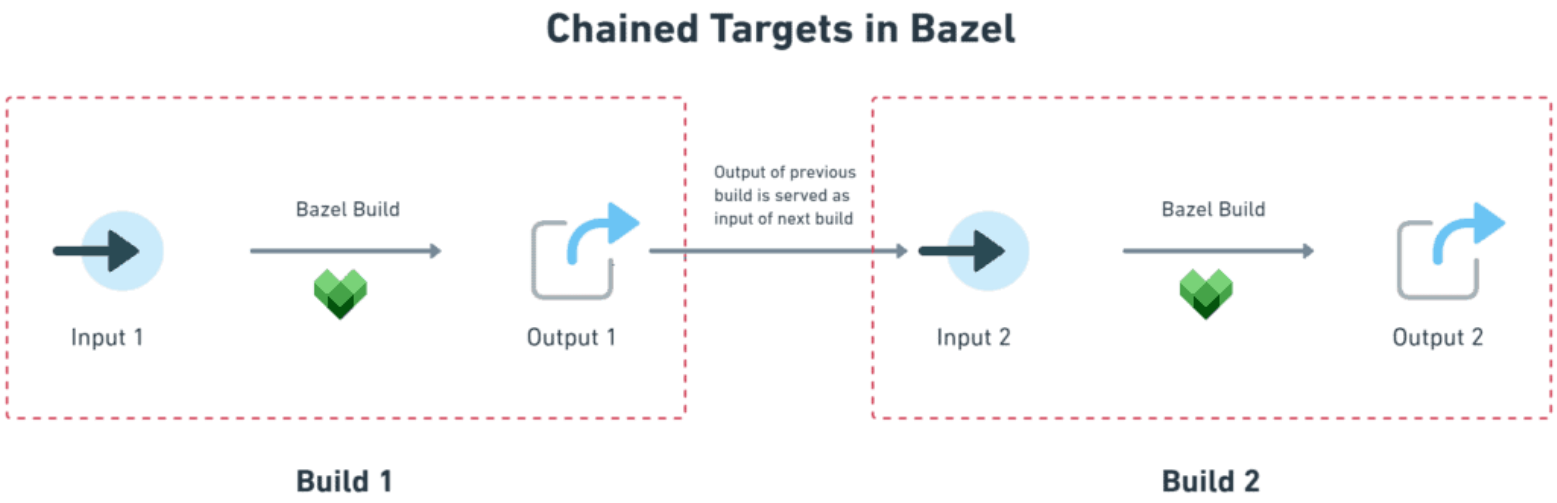
```
load("@rules_cc//cc:defs.bzl", "cc_binary")

cc_binary(
    name = "hello-world",
    srcs = ["hello-world.cc"],
)
```

Bazel Terminology contd..

Targets

Everything inside your packages can be considered targets. It includes source files that developers of your team wrote and added to your project or generated files that Bazel constructs based on your build configurations.



Bazel Terminology contd..

Labels

The nomenclature of a target is known as a label. It's just a way to recognize different targets falling under a package and distinguishing them from other targets in the same or different packages.

Consider the following label:

@repo//application:application_source

The above label describes the target application_source inside a package application. Similarly, in the below example main_binary describes a target inside a subpackage main under the package application.

Bazel Terminology contd..

Dependencies

When you build your specified targets, one or more target may depend on another for the build process. The latter is a dependency. To explain further, consider two targets Target1 and Target2. Let's say Target1 needs Target2 at build or execution time. We can express this relationship as a Directed Acyclic Graph that demonstrates how Target1 is dependent on `Target12 in this process. Bazel calls this a dependency graph and is used to differentiate between actual and declared dependencies.

Bazel Terminology contd..

Commands

Bazel offers a set of commands that allow you to execute certain operations.

The simplest command you can run is to check if Bazel is successfully installed in your system:

$ bazel –version

Other important commands are bazel build, bazel run and bazel test. The build command as the name suggests builds your output files for the specified inputs or targets.

$ bazel build //:MyProject

Bazel Installation

NobleProg

HITESH KUMAR SHARMA

Bazel Installation

Check your system

Recommended: 64 bit Windows 10, version 1703 (Creators Update) or newer

To check your Windows version:

•Click the Start button.
•Type ***winver*** in the search box and press Enter.
•You should see the About Windows box with your Windows version information.

Bazel Installation

Install the prerequisites

- Visual C++ Redistributable for Visual Studio 2015

- Download the Bazel binary (bazel-version-windows-x86_64.exe) from GitHub.
- Install Bazel from Chocolatey

Bazel Installation

Set up your environment

To make Bazel easily accessible from command prompts or PowerShell by default, you can rename the Bazel binary to bazel.exe and add it to your default paths.

set PATH=%PATH%;path to the Bazel binary

You can also change your system PATH environment variable to make it permanent.

Done
"Success: You've installed Bazel.“

To check the installation is correct, try to run:

bazel *version*

END OF PART-1

THANK YOU

NobleProg

HITESH KUMAR SHARMA